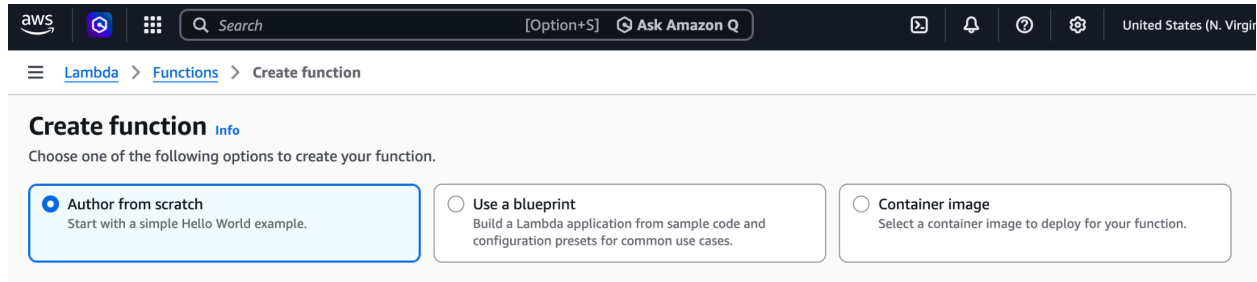


AWS Hands-On Notes

AWS Lambda

Setting up an HTTP Endpoint with Lambda

When setting up a Lambda function to serve an HTTP endpoint (like returning an HTML page, as mentioned), the following steps and considerations are essential:



1. Function Configuration

- **Runtime:** Select the appropriate runtime (e.g., Node.js 20.x).
- **Handler:** This specifies the file and function within your deployment package that AWS Lambda executes. For Node.js, this is usually `index.handler` (meaning the `handler` function in the `index.js` file).

aws

Search

[Option+S]

Ask Amazon Q

United States (N. Virgin

Lambda > Functions > Create function

Basic information

Function name

Enter a name that describes the purpose of your function.

shankfunc

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime

Info

Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

nodejs24.x

Search Runtime

Latest supported

.NET 10 (C#/F#/PowerShell)

Java 25

Node.js 24.x

Python 3.14

Ruby 3.4

Amazon Linux 2023

OS-only runtime for Go, Rust, C++, custom

Other supported

.NET 8 (C#/F#/PowerShell)

Java 11

Java 17

Java 21

ons. Supports Python and Node.js runtimes. [View pricing](#)

u can customize this default role later when adding

Use additional configurations to set up networking, security, and governance for your function. These settings help secure and customize your Lambda function deployment.

2. Code Structure (Node.js Example)

To return a webpage, your handler function must return a response object that follows the format expected by the API Gateway or Function URL (the typical triggers for HTTP requests).

Lambda > Functions > shankfunc

Successfully created the function shankfunc. You can now change its code and configuration. To invoke your function with a test event, choose "Test".

shankfunc

Throttle Copy ARN Actions

Function overview Info

Diagram Template



+ Add trigger

+ Add destination

Description

Last modified
25 seconds ago

Function ARN

arn:aws:lambda:us-east-1:589159458470:function:shankfunc

Function URL Info

-

Export to Infrastructure Composer

Download

Code Test Monitor Configuration Aliases Versions

Code source Info

Open in Visual Studio Code Upload from



Lambda > Functions > Create function

Successfully created the function shankfunc. You can now change its code and configuration. To invoke your function with a test event, choose "Test".

This function contains external libraries.

```
1 import * as fs from 'node:fs';
2
3 const html = fs.readFileSync('index.html', { encoding: 'utf8' });
4
5 /**
6  * Returns an HTML page containing an interactive Web-based tutorial.
7  * Visit the function URL to see it and learn how to build with lambda.
8  */
9 export const handler = async () => {
10   const response = {
11     statusCode: 200,
12     headers: {
13       'Content-Type': 'text/html',
14     },
15     body: html,
16   };
17   return response;
18 };
19
```

1:1 JavaScript

Function URL

Use function URLs to assign an HTTP endpoint to your function.

The HTTP endpoint you create is publicly available, meaning that any unauthenticated user that has the URL can invoke your function. You can set stricter access controls after you create the function. If from this function exceeds free tier allowance, you will be billed accordingly. For more information, see Security and auth model for Lambda function URLs and Lambda Pricing.

Acknowledgment

I understand that the URL which invokes my function is accessible to anyone that has the link and that AWS will bill me for usage that exceeds free tier.

Info Tutorials

Create a simple web app

Task 1: Create the function

Total steps: 5

- Step 1: Choose "Use a blueprint"
- Step 2: Choose the blueprint
- Step 3: Name your function
- Step 4: Acknowledge public Function URL statement
- Step 5: Complete function creation

Task 2: Invoke the function

Total steps: 2

Dismiss tutorial

Task 1: Create the function
Choose Create function. This deploys the code.
Step 5/5 Previous Next

Cancel

Create function

JavaScript

```
exports.handler = async (event) => {
  const htmlContent = `
    <!DOCTYPE html>
    <html>
    <head>
      <title>My Lambda Webpage</title>
    </head>
    <body>
      <h1>Hello from AWS Lambda!</h1>
      <p>This page was served by a Node.js function.</p>
    </body>
    </html>
  `;

  const response = {
    statusCode: 200,
    headers: {
      'Content-Type': 'text/html', // Crucial for rendering HTML
      // ...
    },
    body: htmlContent,
  };
  return response;
};
```

3. Trigger Configuration

To make your function accessible via a URL, you need a trigger:

Function URL | [Info](#)

 <https://jiisaerqk26cqcqgidfeomyy240ihwzb.lambda-url.us-east-1.on.aws/> 

- **Lambda Function URL:** The simplest method. This provides a dedicated HTTP(S) endpoint for your function.
 - **Auth Type:** Usually set to **NONE** for public web pages.
 - **CORS:** Configure if the webpage will be accessed from different domains.
- **API Gateway:** Recommended for more complex scenarios, such as needing multiple routes, request validation, authentication, or caching.

- Set up a **Resource** and **Method** (e.g., GET on /).
- Integrate the method with your Lambda function.

4. Blueprints and Templates

As noted, using a blueprint simplifies the setup:

Blueprint/Template	Purpose
Getting Started with HTTP	Provides a basic framework for receiving HTTP requests and returning responses, often pre-configured with an API Gateway or Function URL. Ideal for simple web serving.
Microservice	For handling RESTful APIs, often includes database integration (e.g., DynamoDB).
Custom Function	For functions with specific requirements not covered by other templates.

Hello from AWS Lambda!

The blueprint function you created rendered this page

This is a learning example. Our customers use Lambda to run web and mobile backends, microservices, process streams in real time, run big data processing or ML inference workloads, and much more.

Aditya Shankar has tried AWS Lambda! If you see this, that means the deployment was successful!

Step 1: Read user input in your Lambda function

Step 2: Store the input in a serverless database

Step 3: Render the content from the database